

گزارش پروژه ریاضیات گسسته

# بخش رمز سزار

CAESAR CIPHER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| A | B | C | D | E | F | G |
| D | E | F | G | H | I | J |

شماره دانشجویی

40418153

نویسنده

محمدعلی جعفری

## بخش رمز سزار (Caesar Cipher)

این پوشه پیاده‌سازی رمز سزار را شامل می‌شود. برنامه علاوه بر رمزگذاری و رمزگشایی با کلید معلوم، هر ۲۶ کلید را امتحان می‌کند، محتمل‌ترین متن را با روش آماری انتخاب می‌کند و فراوانی حرف، دوگرام، سه‌گرام، شاخص تطابق و آنتروپی شانون را به دست می‌آورد. بخش آزمایش نیز اثر طول متن بر دقت رمزشکنی و زمان اجرای روش‌ها را در فایل CSV و نمودار ثبت می‌کند.

در این پیاده‌سازی فقط حروف انگلیسی جابه‌جا می‌شوند. فاصله، عدد، نشانه‌های نگارشی و حروف غیرانگلیسی بدون تغییر می‌مانند و بزرگی و کوچکی حروف انگلیسی نیز حفظ می‌شود. برای نمونه، رمزگذاری `Hello, World! 123` با کلید ۳ برابر `Khoor, Zruog! 123` است.

### روش پیاده‌سازی

منطق برنامه به سه بخش جدا تقسیم شده است:

- `src/caesar.py` عملیات اصلی رمز، نرمال‌سازی کلید و تولید ۲۶ حالت Brute Force را انجام می‌دهد.
- `src/analysis.py` شامل فراوانی حروف، Chi-Square، دوگرام و سه‌گرام، شاخص تطابق و آنتروپی شانون است.
- `src/experiment.py` نرخ موفقیت و زمان اجرا را اندازه می‌گیرد و فایل‌های CSV و نمودارها را می‌سازد.

روش اصلی انتخاب خودکار کلید، آزمون هر ۲۶ حالت و کمینه‌کردن امتیاز Chi-Square است. روش ساده n-gram نیز برای مقایسه وجود دارد. تفاوت این دو با Brute Force معمولی در این است که Brute Force فقط همه خروجی‌ها را نشان می‌دهد، ولی Chi-Square و n-gram از میان همان ۲۶ خروجی یک مورد را به‌عنوان پاسخ محتمل انتخاب می‌کنند.

## ساختار فایل‌ها

- رمزگذاری، رمزگشایی، نرمال‌سازی کلید و Brute Force: `src/caesar.py`
- شمارش فراوانی، Chi-Square، تحلیل n-gram، شاخص تطابق و آنتروپی: `src/analysis.py`
- آزمایش نرخ موفقیت، بنچمارک زمان اجرا و تولید خروجی‌ها: `src/experiment.py`
- منوی تعاملی و رابط خط فرمان: `main.py`
- تست‌های خودکار رمز و توابع آماری: `tests/test_caesar.py`
- پیکره نمونه انگلیسی: `data/input/english_sample.txt`
- محل گزارش‌های متنی و CSV: `data/output/`
- نتیجه اندازه‌گیری زمان اجرا: `benchmarks/results.csv`
- نمودارهای نرخ موفقیت، زمان اجرا و فراوانی حروف: `docs/images/`
- گزارش فارسی پروژه: `docs/report.md`
- نسخه LaTeX گزارش: `docs/report.tex`
- وابستگی‌های تست و رسم نمودار: `requirements.txt`

## مدلسازی ریاضی و اثبات‌ها

### ۱. نمایش الفبا در حساب پیمانه‌ای

حروف انگلیسی را با اعضای مجموعه زیر نمایش می‌دهیم:

```
Z_26 = {0, 1, 2, ..., 25}
```

نگاشت حروف به اعداد به شکل  $A = 0$ ،  $B = 1$ ، ... و  $Z = 25$  است. اگر عدد متناظر یک حرف  $x$  و کلید عدد صحیح  $k$  باشد، کلید مؤثر برنامه برابر است با:

```
k_bar = k mod 26
```

به همین دلیل کلیدهای ۳، ۲۹ و -۲۳ رفتار یکسانی دارند:

```
29 mod 26 = 3
-23 mod 26 = 3
```

### اثبات همارزی کلیدها

دو کلید  $k_1$  و  $k_2$  زمانی یک تبدیل یکسان می‌سازند که اختلافشان مضربی از ۲۶ باشد:

```
k_1 ≡ k_2 (mod 26)
```

در این صورت عدد صحیحی مانند  $t$  وجود دارد که  $k_1 = k_2 + 26t$ . بنابراین:

```
(x + k_1) mod 26
= (x + k_2 + 26t) mod 26
= (x + k_2) mod 26
```

پس بی‌نهایت کلید صحیح به تنها ۲۶ کلاس همارزی تبدیل می‌شوند و نرمال‌کردن کلید چیزی از اطلاعات لازم برای رمزگذاری کم نمی‌کند.

## ۲. فرمول رمزگذاری و رمزگشایی

برای هر حرف متن اصلی با مقدار عددی  $x$  داریم:

```
Encryption:  $E_k(x) = (x + k) \bmod 26$ 
Decryption:  $D_k(y) = (y - k) \bmod 26$ 
```

مثلاً برای حرف  $x$  و کلید ۳:

```
X = 23
E_3(23) = (23 + 3) \bmod 26 = 0 = A
```

این همان برگشت از انتهای الفبا به ابتدای آن است.

### اثبات درستی رمزگشایی

خروجی رمزگذاری را داخل تابع رمزگشایی قرار می‌دهیم:

```
D_k(E_k(x))
= ((x + k) - k) \bmod 26
= x \bmod 26
= x
```

تساوی آخر به این دلیل درست است که  $x$  از ابتدا عضوی از  $Z_{26}$  و بین صفر و ۲۵ بوده است. جهت عکس نیز برقرار است:

```
E_k(D_k(y)) = ((y - k) + k) \bmod 26 = y
```

پس  $D_k$  معکوس  $E_k$  است و تبدیل سزار یک تابع دوسویی روی الفباست. در نتیجه برای هر متن و هر کلید صحیح:

```
decrypt(encrypt(text, k), k) = text
```

تست رفت و برگشت پروژه همین خاصیت را برای کلیدهای عادی، منفی و بزرگ‌تر از ۲۶ بررسی می‌کند.

### ● ۳. حفظ قالب متن

فرمول بالا فقط روی حروف انگلیسی اعمال می‌شود. برنامه برای حروف بزرگ نسبت به کد  $A$  و برای حروف کوچک نسبت به کد  $a$  اندیس می‌سازد. هر کاراکتر دیگر مستقیماً در خروجی قرار می‌گیرد. بنابراین:

- تعداد کاراکترهای متن تغییر نمی‌کند؛
- جای فاصله‌ها، عددها و نشانه‌ها ثابت می‌ماند؛
- بزرگی و کوچکی هر حرف حفظ می‌شود؛
- حروف فارسی و سایر نویسه‌های غیرانگلیسی تغییر نمی‌کنند.

اگر طول کل متن را  $n$  بگیریم، هر کاراکتر دقیقاً یک بار بررسی می‌شود؛ پس رمزگذاری و رمزگشایی هر دو زمان  $O(n)$  و برای ساخت خروجی حافظه  $O(n)$  لازم دارند.

### ● ۴. فضای کلید و عملی بودن Brute Force

کلید نرمال‌شده یکی از اعضای  $Z_{26}$  است؛ بنابراین:

```
K = {0, 1, 2, ..., 25}
|K| = 26
```

کلید صفر متن را تغییر نمی‌دهد، اما هنگام جست‌وجو باید بررسی شود؛ چون از نظر ریاضی یک کلید معتبر است. برای آزمون کامل، هر یک از ۲۶ کلید روی متنی با  $n$  کاراکتر اجرا می‌شود. هزینه هر رمزگشایی  $O(n)$  است، پس:

```
T_brute_force(n) = 26 × O(n) = O(26n) = O(n)
```

حذف ضریب ۲۶ در نماد  $O$  به معنی حذف کار واقعی نیست؛ فقط نشان می‌دهد که این ضریب با افزایش طول متن ثابت می‌ماند. تابع `brute_force` هر ۲۶ متن را در یک فهرست نگه می‌دارد، بنابراین حافظه آن  $O(26n)$  و با ثابت بودن اندازه الفبا معادل  $O(n)$  است.

این فضای کلید آنقدر کوچک است که حتی بدون هیچ تحلیل آماری می‌توان همه جواب‌ها را فوراً دید. تحلیل آماری فقط کار انتخاب پاسخ درست از میان این ۲۶ گزینه را خودکار می‌کند.

### ● ۵. فراوانی حروف زیر انتقال سزار

فرض کنید تعداد رخداد حرف  $a$  در متن اصلی  $f_P(a)$  باشد. نگاشت سزار با کلید  $k$  را با  $\pi_k$  نشان می‌دهیم:

```

π_k(a) = (a + k) mod 26

```

چون  $\pi_k$  دوسویی است، هر رخداد  $a$  دقیقاً به یک رخداد  $\pi_k(a)$  تبدیل می‌شود و هیچ حرف دیگری به همان مقصد نمی‌رسد. پس:

```

f_C(π_k(a)) = f_P(a)

```

اگر  $N$  تعداد کل حروف انگلیسی باشد، فراوانی نسبی نیز حفظ می‌شود:

```

p_C(π_k(a)) = f_C(π_k(a)) / N
              = f_P(a) / N
              = p_P(a)

```

بنابراین سزار شکل توزیع فراوانی را عوض نمی‌کند و فقط برچسب ستون‌های آن را می‌چرخاند. مثلاً پرتکرارترین حرف متن اصلی پس از رمزگذاری همچنان همان تعداد رخداد را دارد، ولی با نام دیگری دیده می‌شود. همین باقی‌ماندن الگوی فراوانی پایه حمله آماری است.

تابع `frequency_distance` فاصله  $L1$  توزیع متن از توزیع مرجع انگلیسی را نیز محاسبه می‌کند:

```

distance(P, English) = sum(|p_P(a) - p_English(a)|)

```

مقدار کوچک‌تر یعنی توزیع تک‌حرفی متن به جدول مرجع نزدیک‌تر است.

## ۶. بازیابی کلید با Chi-Square

برنامه ابتدا هر ۲۶ کلید را امتحان می‌کند. برای خروجی هر کلید، تعداد مشاهده‌شده حرف  $a$  را  $O_a$  می‌نامیم. اگر طول متن پاک‌شده  $N$  و احتمال مرجع آن حرف در انگلیسی  $p_a$  باشد، تعداد مورد انتظار برابر است با:

```

E_a = N × p_a

```

امتیاز Chi-Square از رابطه زیر به دست می‌آید:

```
chi_square = sum((O_a - E_a)^2 / E_a)
```

مجموع روی هر ۲۶ حرف محاسبه می‌شود. مربع اختلاف باعث می‌شود کمبود و زیادی فراوانی یکدیگر را خنثی نکنند و تقسیم بر  $E_a$  اختلاف هر حرف را متناسب با فراوانی مورد انتظار آن بسنجد. برنامه کلیدی را انتخاب می‌کند که کمترین امتیاز را داشته باشد:

```
best_key = argmin(k in Z_26) chi_square(decrypt(C, k))
```

در کلید درست، خروجی باید از نظر آماری به متن انگلیسی نزدیک‌تر باشد؛ بنابراین معمولاً اختلاف‌های  $O_a - E_a$  کوچک‌تر می‌شوند. این استدلال یک معیار آماری است، نه تضمین برای هر متن. عبارت کوتاهی مثل یک نام یا یک مخفف ممکن است توزیعی کاملاً متفاوت با میانگین زبان داشته باشد و کلید دیگری امتیاز بهتری بگیرد.

برای هر کلید یک بار متن رمزگشایی و فراوانی آن شمرده می‌شود. با الفبای ثابت ۲۶ حرفی، مرتبه زمانی کل روش چنین است:

```
O(26n + 26*26) = O(n)
```

## ۷. دوگرام، سه‌گرام و روش n-gram

در تحلیل n-gram ابتدا فقط حروف انگلیسی نگه داشته و به حروف بزرگ تبدیل می‌شوند. اگر متن پاک‌شده دنباله  $X_0$ ،  $X_1, \dots, X_{(N-1)}$  باشد، n-gram شروع‌شده از جایگاه  $i$  برابر است با:

```
G_i = X_i X_{(i+1)} ... X_{(i+n-1)}
```

اندیس شروع می‌تواند از صفر تا  $N-n$  باشد؛ پس تعداد کل n-gram‌های همپوشان برابر است با:

```
N - n + 1
```

برای نمونه، دوگرام‌های **ABAB** عبارت‌اند از **AB**، **BA** و **AB**؛ بنابراین شمارش **AB** برابر ۲ است.

روش مقایسه‌ای پروژه برای هر متن کاندید تعداد تعدادی از دوگرام‌های رایج مانند **TH** و **HE** و سه‌گرام‌هایی مانند **THE** و **AND** را با وزن ثابت جمع می‌کند. امتیاز نهایی چنین ترکیب می‌شود:



```
score(P) = 0.35 × bigram_score(P) + 0.65 × trigram_score(P)
best_key = argmax(k in Z_26) score(decrypt(C, k))
```

چرا این روش اطلاعاتی بیشتر از فراوانی تک‌حرفی دارد؟ انتقال سزار ترتیب جایگاه‌ها را تغییر نمی‌دهد. اگر **AB** در متن اصلی چند بار رخ داده باشد، همه آن رخدادها پس از یک انتقال ثابت به یک دوگرام یکسان تبدیل می‌شوند. پس ساختار دنباله‌ها باقی می‌ماند، هرچند نام حروف عوض می‌شود. کلید درست نام حروف را به حالت طبیعی برمی‌گرداند و در نتیجه الگوهای رایج انگلیسی دوباره ظاهر می‌شوند.

فهرست n-gram‌های پروژه کوتاه و آموزشی است، در حالی که جدول Chi-Square برای همه حروف مقدار مرجع دارد. به همین دلیل Chi-Square روش پیش‌فرض برنامه است و n-gram بیشتر برای مقایسه و آزمایش استفاده می‌شود.

## ● ۸. شاخص تطابق (Index of Coincidence)

اگر متن **N** حرف داشته باشد و فراوانی حرف **a** برابر **f\_a** باشد، شاخص تطابق از رابطه زیر محاسبه می‌شود:

```
IC = sum(f_a × (f_a - 1)) / (N × (N - 1))
```

### اثبات فرمول IC

از **N** جایگاه متن،  $N \times (N - 1)$  راه برای انتخاب یک زوج مرتب از دو جایگاه متفاوت وجود دارد. برای حرف **a** نیز  $f_a \times (f_a - 1)$  زوج مرتب داریم که هر دو عضو آن حرف **a** هستند. جمع این تعداد برای همه حروف و تقسیم آن بر تعداد کل زوج‌ها، احتمال همسان‌بودن دو حرف انتخاب‌شده را می‌دهد؛ این احتمال همان IC است.

اگر توزیع حروف کاملاً یکنواخت باشد:

```
IC_uniform = 26 × (1/26)^2 = 1/26 ≈ 0.0385
```

زبان انگلیسی به دلیل توزیع نامساوی حروف معمولاً IC بزرگ‌تری دارد. البته برای متن‌های کوتاه مقدار تجربی می‌تواند نوسان زیادی داشته باشد.

### اثبات ثابت‌ماندن IC در رمز سزار

در بخش فراوانی ثابت شد که رمز سزار فقط مقادیر **f\_a** را میان برچسب‌های الفبا جابه‌جا می‌کند. بنابراین صورت کسر IC بعد از رمزگذاری همان جمله‌های قبلی را با ترتیب دیگری جمع می‌کند:

```
sum(f_C(b) × (f_C(b) - 1))
= sum(f_P(a) × (f_P(a) - 1))
```

تعداد کل حروف  $N$  نیز ثابت است؛ در نتیجه:

```
IC(ciphertext) = IC(plaintext)
```

پس IC می‌تواند نشان دهد متن دارای توزیعی شبیه زبان طبیعی است، اما نمی‌تواند بین ۲۶ انتقال سزار فرق بگذارد. هر ۲۶ خروجی دقیقاً IC یکسانی دارند.

## ۹. آنتروپی شانون

اگر احتمال تجربی حرف  $a$  برابر  $p_a = f_a/N$  باشد، آنتروپی تک‌حرفی متن چنین تعریف می‌شود:

```
H(X) = -sum(p_a × log2(p_a))
```

جمله‌های دارای احتمال صفر در مجموع وارد نمی‌شوند. اگر متن فقط از یک حرف تشکیل شده باشد، احتمال آن حرف ۱ است و داریم  $H = -1 \times \log_2(1) = 0$ . برای الفبای ۲۶ حرفی، بیشترین مقدار آنتروپی در توزیع یکنواخت رخ می‌دهد:

```
H_max = -26 × (1/26) × log2(1/26)
        = log2(26)
        ≈ 4.7004 bits per letter
```

## اثبات ثابت ماندن آنتروپی

چون  $p_C(\pi_k(a)) = p_P(a)$  و  $\pi_k$  یک جایگشت است، رمزگذاری فقط ترتیب جمله‌های مجموع آنتروپی را عوض می‌کند:

```
H(C)
= -sum(p_C(π_k(a)) × log2(p_C(π_k(a))))
= -sum(p_P(a) × log2(p_P(a)))
= H(P)
```

پس سزار آنتروپی تک‌حرفی را افزایش نمی‌دهد. رمز ممکن است در نگاه اول نامفهوم باشد، اما از دید توزیع تک‌حرفی همان مقدار اطلاعات آماری متن اصلی را نگه می‌دارد. فایل‌های `statistics.csv` و `entropy_results.csv` این برابری را برای متن ورودی و نسخه رمزشده با کلید ۵ ثبت می‌کنند.

## ۱۰. مدل آزمایش نرخ موفقیت

طول‌های پیش‌فرض آزمایش عبارت‌اند از:

```
10, 20, 50, 100, 200, 500, 1000
```

برای هر طول `L`، برنامه `T` بار یک قطعه تصادفی `L` حرفی از پیکره انتخاب می‌کند، کلیدی تصادفی از صفر تا ۲۵ می‌سازد، متن را رمز می‌کند و کلید را با روش انتخاب‌شده حدس می‌زند. اگر در `S` آزمایش کلید درست بازیابی شود، نرخ موفقیت تجربی برابر است با:

```
success_rate(L) = S / T
```

مقدار ذخیره‌شده در CSV بین صفر و یک است و در نمودار در ۱۰۰ ضرب می‌شود تا به درصد نمایش داده شود. `seed` پیش‌فرض ۴۲ است؛ در نتیجه انتخاب قطعه‌ها و کلیدهای تصادفی قابل تکرارند.

با افزایش طول متن، نسبت مشاهده‌شده هر حرف معمولاً به احتمال واقعی آن در زبان نزدیک‌تر می‌شود:

```
p_hat_L(a) = f_L(a) / L
p_hat_L(a) -> p(a) as L -> infinity
```

این نتیجه از قانون اعداد بزرگ می‌آید و توضیح می‌دهد چرا Chi-Square معمولاً روی متن بلندتر پاسخ مطمئن‌تری می‌دهد. با این حال افزایش نرخ موفقیت برای تک‌تک طول‌ها تضمین‌شده و یکنواخت نیست؛ چون قطعه‌های تصادفی مختلف می‌توانند ویژگی‌های زبانی متفاوتی داشته باشند.

اگر گزینه `--compare-methods` فعال باشد، همین آزمایش یک بار برای Chi-Square و یک بار برای n-gram انجام می‌شود و هر دو خط در نمودار قرار می‌گیرند.

## ۱۱. اندازه‌گیری زمان اجرا

برای هر طول متن، زمان رمزگذاری، رمزگشایی، Brute Force و رمزشکنی آماری با `time.perf_counter()` اندازه‌گیری می‌شود. اگر یک عملیات `R` بار تکرار و زمان کل آن `t` ثانیه باشد، مقدار ثبت‌شده چنین است:



```
average_time_ms = (t × 1000) / R
```

تکرار اندازه‌گیری اثر نوسان‌های کوتاه سیستم را کمتر می‌کند. انتظار نظری این است که زمان همه روش‌ها تقریباً خطی با طول متن رشد کند. نمودار زمان اجرا این انتظار را به‌صورت تجربی بررسی می‌کند؛ قرار نیست زمان دقیق روی دو رایانه یکسان باشد.

## ● ۱۲. پیچیدگی زمانی و حافظه

در جدول زیر  $n$  طول متن و  $q = 26$  اندازه القیاست:

| عملیات               | زمان          | حافظه اصلی                        |
|----------------------|---------------|-----------------------------------|
| رمزگذاری یا رمزگشایی | $O(n)$        | $O(n)$ برای خروجی                 |
| شمارش و فراوانی حروف | $O(n + q)$    | $O(q)$                            |
| شمارش n-gram         | $O(n)$        | تا $O(n)$ برای الگوهای متمایز     |
| Brute Force          | $O(qn)$       | $O(qn)$ برای نگهداری همه خروجی‌ها |
| انتخاب با Chi-Square | $O(qn + q^2)$ | در پیاده‌سازی فعلی $O(qn)$        |
| انتخاب با n-gram     | $O(qn)$       | در پیاده‌سازی فعلی $O(qn)$        |
| IC و آنتروپی         | $O(n + q)$    | $O(q)$                            |

چون  $q = 26$  ثابت است، همه زمان‌های  $O(qn)$  از نظر رشد مجانبی  $O(n)$  هستند. ضعف امنیتی سزار دقیقاً از همین فضای کلید ثابت و کوچک می‌آید.

## خلاصه روش رمز شکنی

1. برنامه متن رمز را با هر کلید صفر تا ۲۵ رمزگشایی می‌کند.
2. در حالت Brute Force هر ۲۶ متن برای بررسی دستی نمایش داده می‌شوند.
3. در حالت Chi-Square فراوانی هر کاندید با جدول مرجع انگلیسی مقایسه می‌شود.
4. کاندیدی که کمترین Chi-Square را دارد به‌عنوان متن محتمل انتخاب می‌شود.
5. در روش n-gram، امتیاز دوگرام‌ها و سه‌گرام‌های رایج برای هر کاندید محاسبه می‌شود.
6. کاندیدی که بیشترین امتیاز n-gram را دارد انتخاب می‌شود.

## نصب و اجرا

دستورها را داخل پوشه `Caesar-kntu` اجرا کنید. ابتدا وابستگی‌ها را نصب کنید:

```
python -m pip install -r requirements.txt
```

### ● منوی تعاملی

برای اجرای ساده برنامه:

```
python main.py
```

منو امکان رمزگذاری، رمزگشایی، نمایش ۲۶ حالت، شکستن خودکار رمز، تحلیل فراوانی، تحلیل n-gram، محاسبه IC و آنتروپی و اجرای آزمایش‌ها را دارد. متن را می‌توان مستقیم وارد کرد یا از فایل UTF-8 خواند. در عملیات‌هایی که خروجی دارند، نوشتن `-` به جای مسیر باعث می‌شود نتیجه فقط در ترمینال نمایش داده شود.

### ● رمزگذاری و رمزگشایی

```
python main.py encrypt --text "Hello, World!" --key 3
python main.py decrypt --text "Khoor, Zruog!" --key 3
```

خواندن از فایل و ذخیره خروجی:

```
python main.py encrypt --input data/input/english_sample.txt --key 7 --output
data/output/ciphertext.txt
python main.py decrypt --input data/output/ciphertext.txt --key 7 --output data/output/decrypted.txt
```

### ● Brute Force و شکستن خودکار رمز

نمایش همه کلیدها:



```
python main.py brute-force --text "Khoor Zruog!"
python main.py brute-force --input data/output/ciphertext.txt --output data/output/brute_force.txt
```

انتخاب خودکار با Chi-Square:



```
python main.py crack --text "Khoor Zruog!"
python main.py crack --method chi-square --input data/output/ciphertext.txt --output
data/output/recovered.txt
```

انتخاب با دوگرام و سه‌گرام:



```
python main.py crack --method ngram --input data/output/ciphertext.txt --output
data/output/recovered_ngram.txt
```

## ● تحلیل آماری متن

جدول فراوانی حروف:



```
python main.py frequency --text "The quick brown fox"
python main.py frequency --input data/input/english_sample.txt --output
data/output/frequency_analysis.csv
```

نمایش دوگرام‌ها و سه‌گرام‌های پرتکرار:



```
python main.py ngram --input data/input/english_sample.txt
```

محاسبه شاخص تطابق و آنتروپی:



```
python main.py statistics --input data/input/english_sample.txt
```

در منوی تعاملی، گزینه‌های تحلیل علاوه بر نمایش نتیجه می‌توانند فایل‌های استاندارد CSV و نمودار فراوانی را نیز به‌روزرسانی کنند.

## ● اجرای آزمایش‌ها

اجرای پیش‌فرض با ۱۰۰ آزمایش برای هر طول:

```
python main.py experiment
```

انتخاب طول‌ها، تعداد آزمایش، تعداد تکرار اندازه‌گیری زمان و seed:

```
python main.py experiment --lengths 10,50,100,200,500 --trials 100 --runtime-repeats 5 --seed 42
```

مقایسه Chi-Square و n-gram:

```
python main.py experiment --lengths 20,50,100,200 --trials 100 --compare-methods
```

می‌توان متن آزمایش را مستقیم داد یا پیکره دیگری انتخاب کرد:

```
python main.py experiment --text "A sufficiently long English text goes here" --lengths 10,20,30
python main.py experiment --corpus data/input/english_sample.txt --lengths 50,100,200
```

همین آزمایش‌ها مستقیماً از ماژول نیز قابل اجرا هستند:

```
python -m src.experiment --trials 100 --seed 42 --compare-methods
```



## فایل‌های خروجی

- `data/output/ciphertext.txt`: متن رمز تولیدشده در منوی تعاملی
  - `data/output/decrypted.txt`: خروجی رمزگشایی با کلید معلوم
  - `data/output/recovered.txt`: متن بازیابی‌شده به روش آماری
  - `data/output/brute_force.txt`: هر ۲۶ خروجی Brute Force
  - `data/output/success_rate.csv`: نرخ موفقیت برای هر طول و هر روش
  - `data/output/runtime.csv`: میانگین زمان عملیات‌ها برحسب میلی‌ثانیه
  - `data/output/letter_frequency.csv`: تعداد و درصد فراوانی ۲۶ حرف
  - `data/output/bigram_frequency.csv`: دوگرام‌ها، شمارش و فراوانی آنها
  - `data/output/trigram_frequency.csv`: سه‌گرام‌ها، شمارش و فراوانی آنها
  - `data/output/statistics.csv`: مقایسه IC و آنتروپی متن اصلی و متن رمز
  - `data/output/entropy_results.csv`: آنتروپی برحسب بیت برای هر حرف
  - `data/output/cracking_results.csv`: امتیاز Chi-Square تمام کلیدها
  - `benchmarks/results.csv`: نسخه جداگانه نتایج بنچمارک
  - `docs/images/success_rate.png`: نمودار نرخ موفقیت برحسب طول متن
  - `docs/images/runtime.png`: نمودار زمان Brute Force و Chi-Square
  - `docs/images/letter_frequency.png`: نمودار فراوانی حروف
- اگر `matplotlib` نصب نباشد، محاسبات و فایل‌های CSV همچنان ساخته می‌شوند و فقط نمودارها تولید نمی‌شوند. بسته‌های `arabic-reshaper` و `python-bidi` برای نمایش درست نوشته‌های راست‌به‌چپ در نمودارها در نظر گرفته شده‌اند.

## اجرای تست‌ها



```
python -m pytest
```

تست‌ها موارد زیر را پوشش می‌دهند:

- رمزگذاری و رمزگشایی پایه؛
- برگشت از انتهای الفبا به ابتدای آن؛
- حفظ lowercase، uppercase، فاصله، عدد و نشانه‌ها؛
- نرمال‌سازی کلیدهای منفی و بزرگ‌تر از ۲۶؛
- خاصیت `decrypt(encrypt(text, key), key) = text`؛
- تولید دقیق هر ۲۶ خروجی Brute Force؛
- تشخیص کلید یک متن انگلیسی با Chi-Square؛
- شمارش حروف، دوگرام و سه‌گرام؛
- رفتار مشخص توابع آماری برای متن خالی.

## نکات و محدودیت‌ها

- Brute Force سزار همیشه همه حالت‌ها را تولید می‌کند؛ چیزی که ممکن است خطا کند انتخاب خودکار بهترین حالت است.
- جدول فراوانی مرجع، میانگین متن‌های انگلیسی است و الزاماً با یک جمله کوتاه یا متن تخصصی برابر نیست.
- Chi-Square روی متن‌های بلند معمولاً از روش n-gram ساده این پروژه پایدارتر است.
- فهرست دوگرام‌ها و سه‌گرام‌ها محدود است و نقش یک مدل کامل زبان را ندارد.
- IC و آنتروپی در همه انتقال‌های سزار ثابت می‌مانند؛ بنابراین به‌تنهایی برای پیدا کردن کلید مناسب نیستند.
- طول آزمایش برحسب تعداد حروف انگلیسی سنجیده می‌شود، نه تعداد کل کاراکترهای فایل.
- ثابت‌بودن seed نتیجه نمونه‌گیری را تکرارپذیر می‌کند، ولی زمان اجرا به سخت‌افزار و وضعیت سیستم وابسته است.
- این پیاده‌سازی فقط حروف انگلیسی را رمز می‌کند و برای امنیت واقعی مناسب نیست.

پایان گزارش بخش رمز سزار